LESSON 13

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Intruder Alert!

Build a motion-sensing burglar alarm that you can arm and disarm.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **PIR sensor** feels the heat of moving bodies
- How to spot the moment something **changes** (an *edge*)
- How to use a `bool` variable to remember if the alarm is **armed**
- How to tune a sensor with its adjustment knobs

SESSION PLAN

0:00	How do motion sensors work?
0:10	Wire up the PIR sensor and LED
0:25	Sketch 1: **L13_Motion** (+ 1 minute warm-up)
0:40	Knob experiments: range and timer
0:55	Short break
1:00	Add LEDs, button and buzzer
1:15	Sketch 2: **L13_Alarm**
1:35	Challenges: test your alarm on a friend!
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ PIR motion sensor (HC-SR501, white dome)
- ☐ 1 red LED and 1 green LED
- ☐ 2 resistors, 220 Ω (red-red-brown)
- ☐ 1 push button
- ☐ Passive buzzer
- ☐ Jumper wires (3 female-to-male for the PIR)

1. How does it see you?

Everything warm gives off invisible **infrared** light. People, cats and dogs glow brightly in infrared; walls and tables are dim. The **PIR** (Passive InfraRed) sensor has a heat-sensitive chip under its white dome.

The dome is a clever lens made of lots of little windows. When you walk past, your warm body moves from one window to the next, and the sensor notices the *change*. That is why it spots **moving** people but not a person who sits perfectly still.

> **Big idea: the sensor does the hard work**
>
> The PIR sensor has its own little chip. It sends the ESP32 one simple signal: pin **HIGH** = "*I saw movement!*", pin **LOW** = "*all quiet*". Our code just has to read one pin.

2. Build the motion detector

Take the PIR sensor and carefully lift off the white dome to have a look (then put it back!). Find the 3 pins and their labels: **VCC**, **OUT**, **GND**.

Turn the sensor over. You'll find **two orange knobs** (use a small screwdriver or your fingernail):

- One is **SENSITIVITY**: how far away it can see (about 3 to 7 metres).
- One is **TIME**: how long OUT stays HIGH after it sees you (about 3 seconds to 5 minutes!).

Some boards have a little **jumper** too. If yours does, put it on **H** ("repeat"), so the timer keeps restarting while you keep moving.

✓	From	To
☐	ESP32 **GND**	breadboard **–** rail
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	PIR **VCC**	**+** rail (5 V)
☐	PIR **GND**	**–** rail
☐	PIR **OUT**	ESP32 pin **27**
☐	ESP32 pin **26**	220 Ω resistor → **long leg** of the **green** LED
☐	short leg of the green LED	**–** rail

> **The PIR is happy with 5 V**
>
> It needs 5 V to power itself, but its **OUT** pin only ever sends 3.3 V. So it is safe to connect straight to the ESP32, with no divider needed.

3. Sketch 1: Motion detector

> **Wait one minute!**
>
> When it first gets power, the PIR sensor needs about **1 minute** to learn what the room looks like. During this time it may trigger randomly. Keep still and be patient!

File → New Sketch, type this in, save it as **L13_Motion**, upload, and open the **Serial Monitor**.

Type this in → save it as **L13_Motion** — 32 lines

```
// Lesson 13: Motion detector
// PIR sensor on pin 27, LED on pin 26.

const int PIR_PIN = 27;
const int LED_PIN = 26;

int count = 0;
bool wasMoving = false;

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.println("Warming up... keep still for 1 minute!");
}
```

```
void loop() {
  bool moving = digitalRead(PIR_PIN) == HIGH;
  digitalWrite(LED_PIN, moving);

  if (moving && !wasMoving) {       // it just started
    count = count + 1;
    Serial.print("Motion! Count: ");
    Serial.println(count);
  }
  if (!moving && wasMoving) {       // it just stopped
    Serial.println("All quiet.");
  }

  wasMoving = moving;
  delay(50);
}
```

How it works

Code	What it means
`bool wasMoving = false;`	A `bool` holds only **true** or **false**. This one remembers what we saw *last time*
`digitalRead(PIR_PIN) == HIGH`	Is the sensor saying "movement"? The answer (true/false) goes in `moving`
`digitalWrite(LED_PIN, moving);`	`true` works like HIGH and `false` like LOW, so the LED copies the sensor
`moving && !wasMoving`	`&&` means **AND**, `!` means **NOT**. *Moving now* AND *not moving before* = it **just started**
`wasMoving = moving;`	Remember this reading for the next time round the loop

> **Big idea: catching the *edge***
>
> The sensor stays HIGH for a few seconds. If we added 1 to `count` *every time* we saw HIGH, it would count hundreds per wave! Instead, we only count at the **edge**: the moment it changes from LOW to HIGH.

4. Tune your sensor

Remember the two orange knobs underneath the sensor? Now that your sketch is running, find out which knob is which.

Detective work

1. Turn **one** knob fully anticlockwise. Wave, then stand still. How long does the LED stay on? Which knob did you turn?

   ---

2. Set the TIME knob to its **shortest** setting (you'll need it short for the alarm). How far across the room can the sensor see you?

   ---

3. After the LED goes off, the sensor ignores you for about 3 seconds. Can you find this "blind" moment?

   ---

5. Sketch 2: Burglar alarm

Now let's make a real alarm. The **button** arms and disarms it. **Green** = disarmed (safe), **red** = armed. If someone moves while it's armed, the siren wails!

Add these parts:

✓	From	To
☐	ESP32 pin **25**	220 Ω resistor → **long leg** of the **red** LED
☐	short leg of the red LED	**–** rail
☐	ESP32 pin **13**	one leg of the **button**
☐	the button's **opposite** leg	**–** rail
☐	Buzzer **+**	ESP32 pin **33**
☐	Buzzer **–**	**–** rail

File → **New Sketch**, type this in and save it as **L13_Alarm**.

Type this in → save it as **L13_Alarm** — 65 lines

```
// Lesson 13: Burglar alarm
// Button on 13 arms/disarms. Red LED 25, green LED 26, buzzer 33.

const int PIR_PIN = 27;
const int BUTTON_PIN = 13;
const int RED_PIN = 25;
const int GREEN_PIN = 26;
const int BUZZER_PIN = 33;

bool armed = false;
bool wasMoving = false;
int alarms = 0;

void showState() {
  digitalWrite(RED_PIN, armed);
  digitalWrite(GREEN_PIN, !armed);
```

```
  if (armed) {
    Serial.println("ARMED");
  } else {
    Serial.println("Disarmed");
  }
}

void siren() {
  for (int i = 0; i < 6; i++) {
    digitalWrite(RED_PIN, HIGH);
    tone(BUZZER_PIN, 1200);
    delay(250);
    digitalWrite(RED_PIN, LOW);
    tone(BUZZER_PIN, 800);
    delay(250);
  }
  noTone(BUZZER_PIN);
  showState();
}

void setup() {
  Serial.begin(115200);
  pinMode(PIR_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(RED_PIN, OUTPUT);
  pinMode(GREEN_PIN, OUTPUT);
  showState();
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    armed = !armed;
    showState();
    tone(BUZZER_PIN, 2000, 100);
    while (digitalRead(BUTTON_PIN) == LOW) {
      delay(10);                      // wait for the finger to lift
    }
  }

  bool moving = digitalRead(PIR_PIN) == HIGH;
  if (armed && moving && !wasMoving) {
    alarms = alarms + 1;
    Serial.print("INTRUDER! Alarm number ");
    Serial.println(alarms);
    siren();
  }
  wasMoving = moving;
  delay(20);
}
```

Code	What it means
`armed = !armed;`	Flip it: true becomes false, false becomes true
`digitalWrite(GREEN_PIN, !armed);`	Green is on when the alarm is **not** armed
`void siren() { … }`	Our own function: flash red and switch between two notes, 6 times
`while (digitalRead(BUTTON_PIN) == LOW)`	Keep waiting **while** the button is still held down
`armed && moving && !wasMoving`	Armed, AND moving now, AND not moving before

The alarm goes off as soon as I arm it!

You're too close and moving! Arm it, then **freeze** until the PIR's timer runs out. Or point the sensor away from you. (Challenge 2 fixes this properly.)

6. Challenges

Challenge 1: Silent alarm

Make a "spy mode" where the alarm only flashes the red LED and prints a message, with no sound, so the intruder doesn't know they've been spotted.

Challenge 2: Exit delay

Real alarms give you time to leave. After arming, beep once a second for 10 seconds before the alarm becomes active. *Hint: a* `for` *loop with* `tone()` *and* `delay(1000)`*.*

Challenge 3: Alarm diary

When the alarm goes off, print how long after arming it happened, like `Alarm 2 at 01:35`. *Hint: save* `millis()` *when you arm it, then subtract.*

Bonus: Test it on a friend

Hide your alarm guarding a "treasure". Can a friend sneak up and grab it without setting it off? What tricks work against a PIR sensor?

7. Quiz

Check what you learned

1. Why does the PIR see a walking person but not a statue of a person?

2. What does `!` mean in code? What is `!true`?

3. Why do we count motion only when `moving && !wasMoving`?

Tidy up: unplug the USB cable. Put the PIR sensor's dome back on if you took it off.